第 4-4 讲 • 出错怎么办：读懂报错

> **模块 4 • 学会改代码** ｜ 建议用时 **40 分钟**（阅读 20 + 动手 20）
> 前置要求：完成 4-1 ~ 4-3（你已经改过代码，见过报错）

本节目标

读完并做完这一讲，你能：

1. 把"报错"从**恐吓**变成**线索**——知道它是电脑在跟你说话，只是说话很难听。
2. 认出**五种最常见报错**，并知道每一种的**第一反应**是什么。
3. 掌握一套**排查顺序**：从最近的改动开始查，而不是到处乱翻。
4. 学会读**三段式报错**：错误类型 + 位置 + 原因。
5. 建立正确的**心态**：报错是正常现象，不是"我不行"的证据。

一、先破除恐惧

1.1 报错是最"诚实"的东西

现实比喻：报错就像**体检报告**。

- 它不会安慰你，也不会拐弯抹角
- 它说的每个字都是真的
- **它指出问题的位置，但不一定指出根本原因**（就像"转氨酶偏高"没说是因为熬夜还是喝酒）

关键认知：

> **报错不是"你失败了"，而是"这里有个具体问题，位置是 XXX"。**

对比一下：如果程序**不报错但结果是错的**，那才是真正的灾难——你连从哪查都不知道。**所以报错是好事。**

1.2 一个真实的心态数据

在这个项目里，我（写这套讲义的人）每天都在看报错。 举个真实例子：

做讲义 PDF 流水线时，我遇到一个报错：

```
ERR_FAILED (-2) loading 'file:///C:/Users/34123/Desktop/%E8%AE%B0%E8%B4%A6%E7%B0%BF/...'
```

我猜了三次都错了：

1. 以为是中文路径问题 → 单独测中文文件名，能加载 ✅
2. 以为是特殊字符问题 → 换成纯 ASCII，还是失败 ❌
3. 以为是文件名太长 → 缩短，还是失败 ❌

最后实测发现：是**在同一个程序里连续开关窗口**导致的冲突——**单独跑那一份文档完全正常（493.5 KB）**。

解决方案：一份文档用一个干净的新进程。

这个例子说明：

- **报错给的线索是真的，但你的第一直觉可能是错的**
- **要靠"做实验"缩小范围，而不是靠猜**
- **专业和业余的差别不是"不遇到报错"，而是"更快缩小范围"**

二、五种最常见的报错

2.1 第一种：类型错误（`error TSxxxx`）

长相（真实输出）：

```
src/main/billshot-probe.ts(157,39): error TS2322: Type 'number' is not assignable to type
```

逐段拆解：

片段	意思
`src/main/billshot-probe.ts`	**哪个文件**
`(157,39)`	**第 157 行，第 39 个字符**
`error TS2322`	错误编号（TS = TypeScript）
`Type 'number' is not assignable to type 'string'`	**原因**：数字不能赋给"文字"类型

翻译成人话："你在第 157 行，把一个数字塞进了一个只能放文字的位置。"

第一反应：

① 打开那个文件，跳到那一行
② 看那一行在干什么
③ 问自己："这里应该是数字还是文字？"
④ 改对其中一边（改值，或改类型定义）

这类错误的特征：**报错位置极其精确**，照着去看就行。**它是最容易解决的报错。**

注意"一串报错"的情况：有时候改一处类型，会冒出 20 个报错（3-3 讲实测过：加一个字段报 24 条）。

这时候不要慌：**它们通常是同一个原因。**改完根本原因，全部消失。

2.2 第二种：找不到模块（`Cannot find module`）

长相：

```
Error: Cannot find module 'better-sqlite3'
```

翻译："我需要一个叫 `better-sqlite3` 的东西，但找不到。"

第一反应（按顺序试）：

顺序	检查	命令
1	**依赖装了吗**	`Test-Path node_modules`
2	没装 → 装	`npm install --ignore-scripts`
3	装了还报 → 是不是**没构建**	`npm run build`
4	还报 → **看 import 路径写错没**	检查那行的 `from '...'`

这类错误的常见原因：**跳过了"装依赖"这一步**（第 0-2 讲讲过的三件必需品）。

2.3 第三种：命令不存在（`无法将 xxx 识别为`）

长相：

```
无法将"npm"项识别为 cmdlet、函数、脚本文件或可运行程序的名称。
```

翻译："我不知道 `npm` 是什么。"

第一反应：

原因	怎么确认	怎么修
没装那个程序	`node -v` 试试	去装 Node.js（0-2 讲）
装完没重开终端	关掉终端，重新打开，再试	**这个原因最常见**
命令拼错了	对照讲义再打一遍	改正拼写

为什么"重开终端"能解决：环境变量（PATH，告诉系统"去哪些文件夹找程序"）在终端启动时读取。**装完新程序后，老终端不知道。**

2.4 第四种：业务错误（中文提示）

长相（这是记账簿真实的报错文案）：

```
Error: 该分类已归档，请重新选择
```

注意它和前面三种的区别：

	技术错误	业务错误
语言	英文 + 错误码	**中文，人话**
谁写的	编译器 / 系统	**项目开发者（我）**
意思	"代码有问题"	**"这个操作不允许"**
该做什么	改代码	**改操作**（不是 bug！）

翻译：这不是程序坏了，**是程序在按规则拒绝你。**

第一反应：

```
① 这是"我做错了"还是"代码写错了"？
② 如果是业务规则 → 按提示改操作（比如换个分类）
③ 如果你认为规则不对 → 那才需要改代码（去 ipc 层的 validate 里改）
```

这些中文提示从哪来：`src\main\ipc\records.ts` 里的 `validate()` 函数（3-2 讲带你看过），每条规则一句中文。

为什么用户看到的是干净的中文：因为 `toastError` 会剥掉技术前缀：

```
raw.replace(/^Error invoking remote method '[^']+':\s*/, '')   // 删掉 "Error invoking remo
   .replace(/^Error:\s*/, '')                                    // 再删掉 "Error: "
```

如果没有这个处理，用户会看到：

```
Error invoking remote method 'records:create': Error: 该分类已归档，请重新选择
```

这就是"好的错误处理"：技术细节留在开发者这边，用户只看业务信息。

2.5 第五种：验证失败（`FAIL`）

长相：

```
FAIL  迁移后字段缺失  -  期望 tags 字段，实际没有
===== 数据层自检：87/88 通过 =====
```

这不是"程序崩溃"，而是"检查没通过"。

翻译："我按 88 条规则检查了一遍，有 1 条不符合预期。"

第一反应：

```
① 看 FAIL 那一条的说明（它写了"期望什么、实际什么"）
② 问：是我刚改的东西导致的吗？
③ 是 → 修你的改动；不是 → 可能是原本就有的问题（所以要"确认基线"）
```

实用技巧——只筛出失败的：

```
npm run selftest 2>&1 | Select-String "FAIL"
```

输出为空 = 全部通过。 比翻一大段输出快得多。

三、排查顺序（最重要的一节）

3.1 从最近的改动开始

这是最高效的排查原则：

> **先怀疑你刚改的东西，再怀疑别的。**

为什么：

假设	概率
"我改的那行有问题"	**高**（尤其当你刚改完）
"项目里某处用了三年的代码突然坏了"	**极低**

具体做法：

```
① 我刚改了什么？（3–5 个文件）
② 把改动还原，问题还在吗？
   · 还在 → 不是我改的（回到基线问题）
   · 没了 → 就是我改的 → 逐个还原一半，二分查找
③ 缩小到那一处，再仔细看
```

第 ② 步的"二分查找"：改了 4 个文件出问题？**先还原前 2 个**，看是否恢复。这样 2 次就能定位到具体文件。

这就是 4-1 讲强调"先确认基线"和"备份"的原因：它们让这个排查方法成为可能。

3.2 完整的排查清单（照着走）

```
① 读报错原文（别跳过，真的读）
        ↓
② 定位位置（文件名 + 行号）
        ↓
③ 判断类型（技术错误？业务错误？验证失败？）
        ↓
④ 是技术错误 → 看那一行在干什么
   是业务错误 → 这是规则，不是 bug
   是验证失败 → 看"期望 vs 实际"
        ↓
⑤ 还搞不清 → 还原最近的改动，看问题是否消失
        ↓
⑥ 还搞不清 → 缩小范围（注释掉可疑代码 / 二分还原）
        ↓
⑦ 卡超过 20 分钟 → 把完整报错记下来，去问人（或问我）
```

第 ⑦ 步很重要：卡 20 分钟就该求助。继续硬撑的收益极低，而且容易把代码改得更乱。

求助时给什么：完整报错文本（不是截图）+ **你刚改了什么。**这两样能让人 10 秒内定位。

3.3 三个反直觉的提醒

① 报错说的地方，不一定是根本原因

比如"找不到 `RecordSort`"报在 `repo.ts` 第 82 行，但**真正的问题在上面的 import 区。**

报错指的是"我用到了一个不存在的东西"，而不是"你忘写 import 了"。 所以要往上看看。

② 修掉一个错，可能冒出十个

这通常是好事：**它们本来就是同一个根因。**改完根因，一起消失。

③ 报错里最不重要的就是那一堆堆栈

```
at Object.<anonymous> (C:\...\node_modules\...\index.js:69:26)
at Module._compile (node:internal/modules/cjs/loader:1544:14)
at Object.<anonymous> (...)
```

这一堆（叫"调用栈"）说明"代码是怎么一路调到这里的"，对初学者基本没用。 只看**第一行**（最上面的错误信息）和**你自己文件的位置**就够了。

四、动手任务

步骤 1：制造一个类型错误，然后读懂它

这是一个安全的实验（只改一行，马上还原）。

① 先备份：

```
cd "C:\Users\34123\Desktop\记账簿"
Copy-Item src\shared\types.ts "$env:TEMP\types-test.ts" -Force
```

② 制造错误：用 VS Code 打开 `src\shared\types.ts`，找到 `RecordQuery` 里的这一行：

```
  limit?: number
```

改成：

```
  limit?: string
```

③ 看报错：

```
npm run typecheck 2>&1 | Select-String "error TS" | Select-Object -First 5
```

预期看到类似：

```
src/main/db/repositories/records.repo.ts(180,xx): error TS2322: Type 'number' is not assig
```

读懂它：

- 文件：`records.repo.ts`（**不是**你改的 `types.ts`！）
- 原因：数字不能赋给文字

④ **思考**：为什么改 `types.ts`，报错却在 `records.repo.ts`？

答案：因为 `repo.ts` 往 `limit` 里传了一个**数字**，而你把类型改成了"必须是文字"，所以**冲突报在使用方。**

这就是"报错位置不一定是根本原因"的实例。

⑤ **还原**：

```
Copy-Item "$env:TEMP\types-test.ts" src\shared\types.ts -Force
npm run typecheck 2>&1 | Select-String "error TS" | Measure-Object | Select-Object -Expand
```

预期：0。

步骤 2：读一遍所有业务错误文案

```
Get-Content src\main\ipc\records.ts -Encoding UTF8 | Select-String "throw new Error"
```

预期看到 15 条中文报错（3-2 讲见过）。

挑三条，回答：这是"技术错误"还是"业务规则"？

报错	类型
金额不能为 0	业务规则（用户操作问题）
账户不存在	业务规则（数据状态问题）
记录类型不合法	业务规则（**但可能暗示代码 bug**——如果类型系统正常工作，不该出现非法值）

这个练习的价值：**学会区分"用户操作错"和"代码有 bug"**。处理方式完全不同。

步骤 3：练习"从最近改动排查"

任务：制造一个问题，然后用"还原改动"的方式定位它。

① 先人为制造一个错误（用 VS Code）：在 `records.repo.ts` 里，把 `ORDER BY r.date DESC` 改成 `ORDER BY r.dat DESC`（少个 `e`，字段名写错）。

② 观察报错：

```
npm run selftest 2>&1 | Select-String "FAIL" | Select-Object -First 3
```

预期：数据层自检会报错（因为查询失败了）。

③ 排查：读报错，找到"没有 dat 这个字段"之类的线索，回到那一行改正。

④ 还原并确认：

```
npm run selftest 2>&1 | Select-String "通过" | Select-Object -Last 1
```

预期：`88/88 通过`。

这个练习让你体验：**错误 → 读报错 → 定位 → 修复 → 确认** 的完整循环。

步骤 4：学会"只看失败"

```
npm run selftest 2>&1 | Select-String "FAIL"
```

预期：没有输出（因为项目现在是好的）。

记住这个命令：以后跑任何验证，都用它筛失败项，**别去翻一大堆 PASS。**

卡住了怎么办

现象	原因	怎么办
报错太多看不过来	通常是同一个根因	只看**第一条**，修完再看
报错指向 node_modules	通常是依赖问题或你的用法不对	先看报错里**你自己文件**的那一行
改了还是同样的报错	可能没存盘，或改错了文件	确认存盘；确认文件路径
报错看不懂	正常（错误信息是给机器写的）	**把报错原文记下来问人**，别硬猜
还原后还报错	可能还原不干净	用 4-1 讲的备份法重新还原

五、自测题

1. 报错是好事还是坏事？为什么？

2. `error TS2322: Type 'number' is not assignable to type 'string'` 说的是什么？第一反应是什么？

3. "业务错误"和"技术错误"有什么区别？处理方式有什么不同？

4. 排查问题时，为什么"从最近的改动开始"最有效？

5. 如果报错太多看不过来，应该先看哪一条？

答案与解析

1. 好事。因为它**指出具体位置和原因。** → 真正可怕的是"不报错但结果是错的"——那时你连从哪查都不知道。**报错是最诚实的反馈。**

2. 说的是"把一个数字塞进了只能放文字的位置"。第一反应：**打开那个文件、跳到那一行、看那一行在干什么、判断这里应该是数字还是文字。** → 这类错误**位置极其精确**，是最容易解决的。

3. 技术错误（英文 + 错误码）：代码有问题，**要改代码。业务错误**（中文人话）：操作不符合规则，**要改操作**（不是 bug）。 → 区分它们很重要：把业务规则当 bug 去"修"，会把软件的规则改坏。

4. 因为概率差异极大：**"我刚改的那行有问题"概率高**，而"用了三年的代码突然坏了"概率极低。 → 具体做法：还原最近的改动，看问题是否消失。**用二分法（先还原一半）能快速定位到具体文件。**

5. 第一条。因为后面的一堆通常是**同一个根因**导致的连锁报错——改完根因，它们一起消失。 → 3-3 讲实测过：加一个字段报 24 条，删掉字段后全部消失。

六、术语表

术语	一句话解释
报错 / Error	程序或工具告诉你"这里有问题"的信息
错误码 / TS2322	错误的编号，可以拿它搜索
调用栈 / Stack Trace	"代码怎么一路调到这里的"，初学者可忽略
技术错误	代码本身的问题，英文信息
业务错误	操作不符合规则，中文提示
FAIL	验证脚本的"这项没通过"
根因 / Root Cause	真正的原因（可能不在报错指向的位置）
二分排查	通过还原一半改动来快速缩小范围
基线	改动前的干净状态（4-1 讲）
环境变量 / PATH	告诉系统去哪里找程序，重开终端才会刷新

七、本讲与项目的对应

- 你查看的文件：`src\main\ipc\records.ts`（15 条业务报错）、`stores\toast.store.ts`（剥前缀逻辑）
- 你做的实验：制造类型错误 → 读懂 → 还原；制造 SQL 字段名错误 → 定位 → 修复
- **本讲不修改功能代码（实验都会还原），不影响你的记账数据**

- 上一讲：4-3 从需求到设计（下） ｜ **下一讲进入模块 5：5-1 那些 verify 脚本在防什么**

八、模块 4 小结

讲	你学到的
4-1	**安全流程六步**：确认基线 → 分析 → 备份 → 改 → 验证 → 收尾
4-2	**需求分析**：三个问题、判断类型、影响面清单
4-3	**动手实现**：跨三层做排序功能，通过验收与回归
4-4	**读懂报错**：五种常见报错、排查顺序、正确心态

模块 4 结束时，你已经能独立改这个软件了——从需求到实现到验收，全流程走通。

模块 5 要解决一个更高级的问题：**怎么保证你的改动没有破坏别的东西？** 这就是那 15 个验证命令存在的意义。

讲义版本：v1 ｜ 对应项目版本：0.1.0